TokenSaver RAG — FINAL LOCKED Architecture [v3]

LOCKED: Cloud = 0 Rs • Embedding = LOCAL • Callback = EMAIL • LLM = Grok Free Only • No Paid Fallback

Stack: Next.js • FastAPI • PostgreSQL (Docker) • Redis (Docker) • Qdrant (Docker) • Ollama/nomic-embed LOCAL •
Email: Free SMTP (Gmail App Password / Resend Free) • 1 Command: docker compose up -d

A ONLINE REQUEST PATH — User Se Lekar Answer Tak

01 — User
Web Chat • Login
Email Verified

→

02 — API Gateway
JWT Auth • Rate Limit
20 req/min • FastAPI

→

03 — Input Guardrails
Injection • PII • Scope
BLOCK if jailbreak

→

04 — Semantic Cache
Redis • Similarity >0.92
HIT = 0 Token

→

05 — Router + Orchestrator
Intent • Token Budget
Step Limits

→

06 — Output Guardrails
Grounding • Privacy
Validate → Stream

B CONTROLLED EXECUTION — Local Embedding + Grok Only + Email Callback

Retrieval / RAG
Qdrant Self-Host Docker
Hybrid → Re-Rank Top 10→3
Filter: user_id + doc_id

→

PDF + Local Embedding
PDF → Chunk 800 / 100
LOCAL Embedding
Ollama / nomic-embed

→

Grok LLM — Main Only
Grok Free Tier Only
No Paid Fallback
Retry x2 → Queue

→

Email Callback [NEW]
Fail → Queue → Retry
Answer Ready → EMAIL
Q + A + Chat Link

Qdrant • Docker Local
Chunks + Vectors • localhost:6333

PostgreSQL • Docker Local
Users • Docs • Chats • Token Logs

Redis • Docker Local
Cache + Queue + Rate Limit

Email • Free SMTP
Gmail App Password / Resend Free

Observability
Logs • Tokens • Cache HIT/MISS • Latency
PostgreSQL me har request log

Evaluation
Golden Set 20 Q • Recall@10
Grounding Check

Reliability
Retry • Queue • Email Callback
Grok Down → Email se Answer

Release
Docker Compose • Backup
Local → Baad Me Cloud (Jab Paisa Ho)

Knowledge Ingestion (Local)

Email Callback Flow

Recall Check (Baad Me Decide)

TokenSaver RAG — Final Locked v3 Details (Coding Ke Liye)

Is PDF ke baad koi architecture change nahi. Ab seedha coding hogi — Milestone 1 se shuru.

1. Final Stack — Locked

- Frontend: Next.js (localhost:3000) • Backend: FastAPI (localhost:8000) • DB: PostgreSQL Docker
- Cache + Queue: Redis Docker • Vector DB: Qdrant Docker (Dashboard localhost:6333)
- Embedding: LOCAL ONLY — Ollama / nomic-embed (No Cloud, No API, No Rate Limit)
- LLM: Grok Free Tier Only — No Paid Fallback • Email: Free SMTP — Gmail App Password / Resend Free
- Total Cost: 0 Rs / Month (Sirf Laptop + Internet)

2. Email Callback — Final Flow (Tera Decision)

Step 1: User puchta hai — 'Price kya hai?' • Login ke time email already saved hai
Step 2: Grok Call Fail (Rate Limit / Down) → Retry 1 (1 sec wait) → Retry 2
Step 3: Dono fail → Question Redis Queue me → User ko chat me: 'Answer Email pe bhej dunga, thoda wait karo'
Step 4: Background Worker (Celery) har 30 sec me check karta hai → 2 min baad phir Grok try karta hai
Step 5: Success hote hi → EMAIL bhejta hai: Subject: 'Tumhare Sawal ka Jawab — TokenSaver RAG'
Email Body me: Question + Answer + Citations (Page No) + Button: 'Chat me Dekho'
Step 6: Chat me bhi answer save ho jata hai, taaki user baad me chat khole to wahan bhi dikhe
Edge: Agar 3 baar bhi fail → Human Ticket banao, user ko 'Support se contact karenge' wala email bhejo.

3. Local Embedding — Kyun Local Rakha?

- Grok Embedding use karte to uska bhi Free Limit khatam ho sakta tha, PDF upload ruk jata.
- Local Embedding se PDF ka Chunking / Embedding kabhi nahi rukega, chahe Grok down ho.
- Privacy bhi better hai — PDF ka data bahar ke server pe nahi jayega, sirf Top-3 Chunks Grok ko jayenge.
- Setup: docker me ollama pull nomic-embed-text → 1 baar download, phir hamesha local.

4. Milestones — Coding Ka Order (Agentic Se Karwayenge)

Milestone 1: Setup — Docker Compose (postgres + redis + qdrant + backend + frontend) + Folder Structure + .env
Milestone 2: Login + PDF Upload — JWT Auth + PDF Upload (20MB) + Queue + Local Embedding + Qdrant Save
Milestone 3: Chat + Cache + RAG — Semantic Cache (Redis) + Hybrid Search + Re-Rank Top 3 + Grok + Output Guardrail
Milestone 4: Email Callback — Retry + Queue + Celery Worker + Free SMTP Email + Chat me bhi save

5. DB Schema — Locked

PostgreSQL: users(id, email, password_hash) • documents(id, user_id, file_name, status) • chat_sessions(id, user_id, doc_id)
messages(id, session_id, question, answer, tokens_used, cache_hit, status: answered/pending_email) • email_jobs(id, message_id, status)
Qdrant: chunks(id, doc_id, user_id, chunk_text, page_no, embedding_vector) • Filter: user_id + doc_id
Redis: exact_cache + semantic_cache + celery_queue + rate_limit

6. Ab Next Step Kya Hai?

Tu is PDF ko ek baar dekh ke 'FINAL DONE' bol de. Uske baad main Milestone 1 ka Agentic Coding Prompt bana dunga.
[Us Prompt ko tu apne Agentic Coding Tool me paste karega aur code ban jayega.]
Uske baad hum Milestone 2, 3, 4 aise hi ek-ek karke karenge. Ek saath sab nahi karenge.